Diversificación de carteras con predicción de correlaciones

Trabajo Práctico Integrador — Lógica y Teoría de Grafos

Integrantes: **Manuel García de la Vega y Santiago Ferrer Petit**
Fecha de entrega: 08/11/2026
Repositorio: https://github.com/santiferrerpetit/diversificacion-carteras

> Todos los números de este informe salen de `data/metricas.json`, `data/evaluacion_canastas.csv` y los notebooks de `notebooks/`, y se regeneran con los scripts indicados al final.

Resumen

Construimos un sistema que agrupa 30 acciones del S&P 500 en **canastas diversificadas**, es decir, conjuntos de acciones en los que ningún par se mueva junto. Un modelo de regresión predice la correlación futura (a 60 días hábiles) de cada par de acciones; esas predicciones definen las aristas de un grafo de conflicto, y un coloreo propio del grafo reparte las acciones en canastas. Una aplicación web (FastAPI y React) permite cambiar el umbral y la fecha de corte y ver cómo cambian el grafo y las canastas.

Resultado principal, dicho con honestidad: el modelo mejora poco a la línea base que usa la correlación de los últimos 90 días (la mejor regresión baja el error absoluto medio un 3 %), y las canastas armadas con el modelo **no** quedan mejor diversificadas que las armadas con la correlación histórica. Sí quedan claramente mejor que las armadas al azar. Además, el grafo armado con el modelo no refleja los cambios de régimen del mercado (en la crisis de 2020 el número de canastas no sube, mientras que con la correlación histórica se triplica). Detalle en las secciones 4, 6 y 8.

---

1. El problema y las decisiones de diseño

1.1 Quién tiene el problema y qué decisión se quiere tomar

Una plataforma de inversiones quiere ayudar a sus clientes a armar carteras realmente diversificadas. Diversificar es no poner todos los huevos en la misma canasta: si dos acciones

suben y bajan juntas, tener las dos casi equivale a tener una sola, y el riesgo no se reduce. Lo que importa entonces es **qué tan parecido se mueven** las acciones de una cartera.

Qué es la correlación. Es un número entre −1 y 1 que mide qué tan parecido se mueven dos acciones, calculado sobre sus retornos diarios (cuánto subió o bajó cada una de un día al siguiente) en una ventana de días. Cerca de 1, suben y bajan juntas; cerca de 0, no tienen relación; negativa, se mueven al revés.

Por qué "comprar acciones de distintos sectores" no alcanza. Es lo que hoy sugiere la plataforma, pero dos acciones de sectores distintos pueden moverse casi igual. Por ejemplo, AAPL (tecnología) y KO (consumo primario) tenían una correlación de 0,17 a fines de enero de 2020 y de 0,78 a fines de marzo del mismo año.

Qué es la correlación histórica. Es la correlación calculada con los retornos de una ventana de días *pasados*: en este trabajo, los últimos 30, 90 o 250 días hábiles (`corr_30d` , `corr_90d` y `corr_250d`). Es lo que se usa habitualmente para decidir qué acciones combinar, y supone que los próximos días se van a parecer a los últimos.

Por qué cambia con el tiempo. La correlación no es una propiedad fija de un par de acciones: depende del contexto del mercado. En una crisis suele haber ventas generalizadas y las acciones tienden a moverse más juntas, sin importar su sector, y por eso la correlación sube justo cuando más se necesita diversificar. En nuestros datos, el promedio de los 435 pares fue de 0,18 el 31/01/2020 (calculado con los 90 días anteriores) y de 0,68 el 30/03/2020, en plena crisis.

Por qué mirar el pasado no alcanza. La correlación histórica no anticipa estos cambios, porque por definición solo conoce lo que ya pasó. Con el corte del 31/01/2020, la histórica decía que el mercado estaba poco correlacionado (0,18), y en los 60 días siguientes la correlación promedio real fue de 0,68. En todo el dataset, la correlación de los últimos 90 días y la de los 60 días siguientes se parecen solo moderadamente (correlación de 0,48 entre ambas). Si se usa un umbral de 0,5, de los pares que lo superaban en los últimos 90 días, el 54 % ya no lo superaba en los 60 días siguientes, y otros 5 506 pares que no lo superaban sí lo superaron después (notebook `01_dataset`).

La decisión. Como la histórica no alcanza, el sistema predice la correlación futura con un modelo, y con esa predicción decide **qué acciones agrupar en una misma canasta y cuántas canastas hacen falta**, de modo que dentro de cada canasta las acciones no se muevan juntas.

1.2 Por qué un grafo

El modelo de regresión solo entrega un número por par de acciones (435 valores $\hat{\rho}(i,j)$ para 30 acciones). Eso no le dice al cliente qué acciones juntar. Hace falta una **partición global** de las

acciones en canastas, y mirar pares sueltos no alcanza: A-B y B-C pueden ser compatibles y A-C no. El coloreo propio de un grafo considera todas las combinaciones a la vez y garantiza que ninguna canasta contiene una arista. Ver la sección 7 para la comparación con alternativas como el clustering.

El planteo es el de un **grafo de conflicto**, igual al de las peceras de la práctica de coloreo (hay elementos que no pueden compartir contenedor y se pide el mínimo de contenedores):

Práctica	**Nuestro problema**
Peces (depredador-presa)	Acciones
Arista = no pueden estar juntos	Arista = correlación predicha > umbral θ
Pecera	Canasta
Mínimo de peceras	χ(G) = mínimo de canastas

1.3 Decisiones de diseño principales

Las decisiones completas, con su justificación, están en `documentacion/decisiones.md`. Las más importantes:

- **Stack:** backend en Python (FastAPI, scikit-learn, pandas) y frontend en React con TypeScript.
- **Datos fijos y reproducibles:** los precios se descargan una sola vez y se guardan en el repositorio, para que el resultado no dependa de que Yahoo Finance siga disponible.
- **Los algoritmos de grafos se implementan a mano.** `networkx` se usa solo en los tests y en un script de comparación, para contrastar los resultados y generar grafos al azar.
- **El umbral θ es un parámetro del usuario**, no un valor fijo, porque no hay un θ "correcto": depende de cuántas canastas se quiera manejar.

---

2. Dataset

2.1 Fuente

Precios diarios de cierre ajustado (con splits y dividendos) de **30 acciones del S&P 500 y del ETF `SPY`**, descargados con `yfinance` (Yahoo Finance) del 2014-01-02 al 2023-12-29 (2516 días hábiles, sin valores faltantes). `yfinance` es una librería de código abierto que usa los endpoints públicos de Yahoo; no es una API oficial ni está afiliada a Yahoo, y su documentación indica uso personal o educativo. La descarga se hizo una sola vez (`backend/scripts/descargar_precios.py`) y el resultado está en `data/precios.csv`.

Como referencia del mercado (para calcular el beta de cada acción y la volatilidad del mercado) usamos `SPY`, un ETF que replica el S&P 500. Lo elegimos porque es muy conocido en el ámbito financiero, tiene mucho renombre y es de los instrumentos en los que más gente invierte. Sus retornos son prácticamente idénticos a los del índice S&P 500 (correlación mayor a 0,99), así que el beta y la volatilidad casi no cambian.

Las 30 acciones cubren los 11 sectores y mezclan pares muy correlacionados (por ejemplo JPM-BAC, XOM-CVX) con pares casi independientes (NEM-NVDA, KO-TSLA), para que el modelo vea todo el rango de correlaciones. La lista y el criterio están en `documentacion/ACCIONES.md`. El sector de cada acción está en una tabla propia, `data/sectores.csv`, armada a mano: `yf.Ticker(t).info["sector"]` es lento y a veces falla.

2.2 Transformaciones

1. **Retornos diarios:** `precio_hoy / precio_ayer − 1`. La correlación debe medir cómo se mueven las acciones y no su nivel de precio.
2. **Fechas de corte:** una cada 20 días hábiles (aproximadamente un mes). Con una por día las ventanas se superponen casi por completo y las filas serían casi copias. Se descartan los cortes sin 250 días de historia previa y los que no tienen 60 días hábiles posteriores. Resultan **111 fechas de corte**, de 2014-12-30 a 2023-09-27.
3. **Pares:** con 30 acciones hay C(30, 2) = 435 pares, cada uno una sola vez (con `accion_id_1 < accion_id_2` en orden alfabético).
4. **Dataset por pares:** una fila por par y fecha de corte, **48 285 filas** (435 × 111), en `data/dataset_pares.csv`. Se genera con `python −m scripts.construir_dataset`.

2.3 Features y por qué

Feature	Qué es	Por qué
`corr_30d`, `corr_90d`, `corr_250d`	Correlación de retornos en las últimas 30, 90 y 250 ruedas	Tres ventanas para captar el comportamiento reciente y el de largo plazo
`mismo_sector`	1 si las dos acciones son del mismo sector	Acciones del mismo sector tienden a moverse juntas
`dif_volatilidad`	abs($\sigma_1 - \sigma_2$) con σ de las últimas 90 ruedas	Acciones con volatilidad muy distinta tienden a moverse menos juntas
`dif_beta`	abs($\beta_1 - \beta_2$), β = cov(acción, SPY) / var(SPY) en 90 ruedas	Mide qué tan distinta es la sensibilidad al mercado

Feature	Qué es	Por qué
`volatilidad_mercado`	σ de `SPY` en las últimas 90 ruedas	En las crisis las correlaciones suben; esta columna le dice al modelo en qué clima está el mercado
`corr_futura_60d` (target)	Correlación de los 60 retornos siguientes al corte	Es lo que se quiere predecir

Decisiones sobre las features:

- **Valor absoluto en las diferencias**, para que el resultado no dependa del orden del par.
- **Ventana de 90 días** para volatilidad y beta: es un punto medio entre ruido (30 días) y lentitud (250 días), y coincide con `corr_90d`, que es la línea base. Es una elección nuestra y otras ventanas serían defendibles.
- **No entran como features** los ids de las acciones ni la fecha: con los ids el modelo podría memorizar pares puntuales, y con la fecha no podría generalizar a períodos futuros.
- **El beta se calcula nosotros** con una regresión rodante contra `SPY`. El `beta` de `yfinance` es un valor actual, y usarlo en fechas pasadas filtraría información del futuro.

Correlación de cada feature con el target en todo el dataset (notebook `01-dataset`): `corr_90d` 0,48; `corr_250d` 0,45; `corr_30d` 0,44; `dif_beta` −0,27; `mismo_sector` 0,22; `dif_volatilidad` −0,15; `volatilidad_mercado` 0,13. Que `corr_90d` ya correlacione tanto con el target anticipa que el baseline sea difícil de superar.

2.4 Anti-leakage

Leakage (fuga de información) es cuando una feature usa por error información del futuro: el modelo parecería muy bueno al entrenar y fallaría en uso real. Para evitarlo:

- Todas las features usan solo retornos con fecha **≤ `fecha_corte`**. Solo `corr_futura_60d` mira hacia adelante.
- **Test unitario** (`test_features_no_usan_datos_posteriores_al_corte`, en `backend/tests/test_dataset.py`): altera todos los precios posteriores a un corte y comprueba que las features de ese corte y de los anteriores no cambian, mientras que el target sí. Se comprobó también que el test **falla** si se introduce una fuga a propósito.
- **Partición temporal con gap** (sección 4.2), para que el target de train no se solape con test.
- **Verificación con datos reales** (notebook `01-dataset`): para un par y una fecha, las features calculadas a mano coinciden con las del dataset y las ventanas de las features terminan antes de que empiece la del target.

---

3. Modelos de regresión

Se comparan los tres modelos pedidos que aplican a regresión, contra una línea base:

Modelo	Hiperparámetros probados
Baseline	Ninguno: predice que la correlación futura es `corr_90d`
Regresión lineal (features estandarizadas)	—
kNN (features estandarizadas)	`k` ∈ {15, 50, 150, 400, 1000}
Árbol de decisión	`max_depth` ∈ {2, 3, 5, 8} × `min_samples_leaf` ∈ {5, 20, 100, 400}

Se usan los algoritmos de `scikit-learn` entrenados con nuestro dataset, y no modelos preentrenados: la tarea es específica (correlación a 60 días entre pares de estas 30 acciones, con estas features) y no existe un modelo ya entrenado para ella. Con una tabla de 7 columnas y unas 48 mil filas, los modelos simples alcanzan y uno grande se sobreajustaría.

3.1 Cómo se entrenó cada modelo

El *algoritmo* viene de `scikit-learn`, pero el *modelo* (lo que aprendió) sale de entrenarlo con nuestro dataset: las 32 625 filas de train, con las 7 features como entrada y `corr_futura_60d` como salida. El entrenamiento está en `backend/scripts/entrenar_modelo.py` (el código de los modelos, en `backend/app/modelo.py`), tarda unos 15 segundos y es reproducible: volver a correrlo da las mismas métricas. El modelo elegido se guarda en `data/modelo.joblib`, que la API carga una sola vez; el archivo no se versiona y se regenera con el script.

- **Regresión lineal:** ajusta un coeficiente por feature, más un término independiente, por mínimos cuadrados. Es el más simple y el más fácil de interpretar.
- **kNN:** casi no tiene entrenamiento en el sentido clásico: guarda los datos de train y, para predecir un par nuevo, busca los `k` pares de train más parecidos según las 7 features y devuelve el promedio de su correlación futura. Lo único que se elige es `k`.
- **Árbol de decisión:** aprende reglas del tipo "`feature` ≤ valor" que van partiendo los datos; `max_depth` y `min_samples_leaf` limitan cuánto puede ajustarse al ruido.

La regresión lineal y el kNN usan las features **estandarizadas** (media 0 y desvío 1), para que ninguna pese más que otra solo por su escala; en el kNN importa porque se calculan distancias entre pares. El modelo que se usa en la aplicación es el de menor MAE en validación temporal, que resultó ser el kNN con `k = 1000` (sección 4.3). Con un `k` tan grande

el kNN promedia muchísimos pares, y por eso sus predicciones quedan muy suavizadas, cercanas al promedio; esto tiene consecuencias para el grafo (sección 6).

---

4. Metodología del modelo y resultados

4.1 Métricas

- **MAE** (error absoluto medio): error promedio en unidades de correlación. Es la métrica principal y la que se usa para elegir hiperparámetros.
- **RMSE:** penaliza más los errores grandes.
- **R^2:** fracción de la varianza que explica el modelo; 0 equivale a predecir siempre la media y un valor negativo es peor que eso.

4.2 Partición de los datos

La partición es **temporal, no aleatoria**: se entrena con el pasado y se evalúa con el futuro.

Parte	Fechas de corte	Filas	Período
Train	75	32 625	2014-12-30 a 2020-11-13
Gap (descartado)	3	1 305	2020-12-14, 2021-01-13, 2021-02-11
Test	33	14 355	2021-03-12 a 2023-09-27

Es aproximadamente 70/30 por fechas de corte.

Por qué no al azar. Con un split aleatorio, un corte del 15 de marzo caería en entrenamiento y uno del 20 de abril en test, y comparten casi los mismos días de datos. Lo comprobamos (notebook `02-split-y-baseline`): el mismo kNN (k = 15) da R^2 = 0,35 con split aleatorio y R^2 = −0,02 con split temporal. El aleatorio es optimista porque el modelo vio filas casi idénticas a las de test.

Por qué un gap. El target de cada fila mira 60 ruedas hacia adelante, así que el target del último corte de train llega hasta febrero de 2021. Sin gap, train y test compartirían información de los mismos días. Los cortes están separados por 20 ruedas y el horizonte es de 60, así que el gap es de 3 cortes. El notebook lo comprueba con los datos reales: con gap 0 se comparten 40 ruedas de target entre el último corte de train y el primero de test; con gap 1, 20; con gap 2 o más, ninguna. Usamos 3, una elección conservadora (el mínimo es 2).

Validación para elegir hiperparámetros. Dentro de train, con 4 folds de **ventana expansiva** (cada fold entrena con el pasado y valida con el bloque siguiente, también con gap). El test no

se usa para elegir nada.

4.3 Resultados

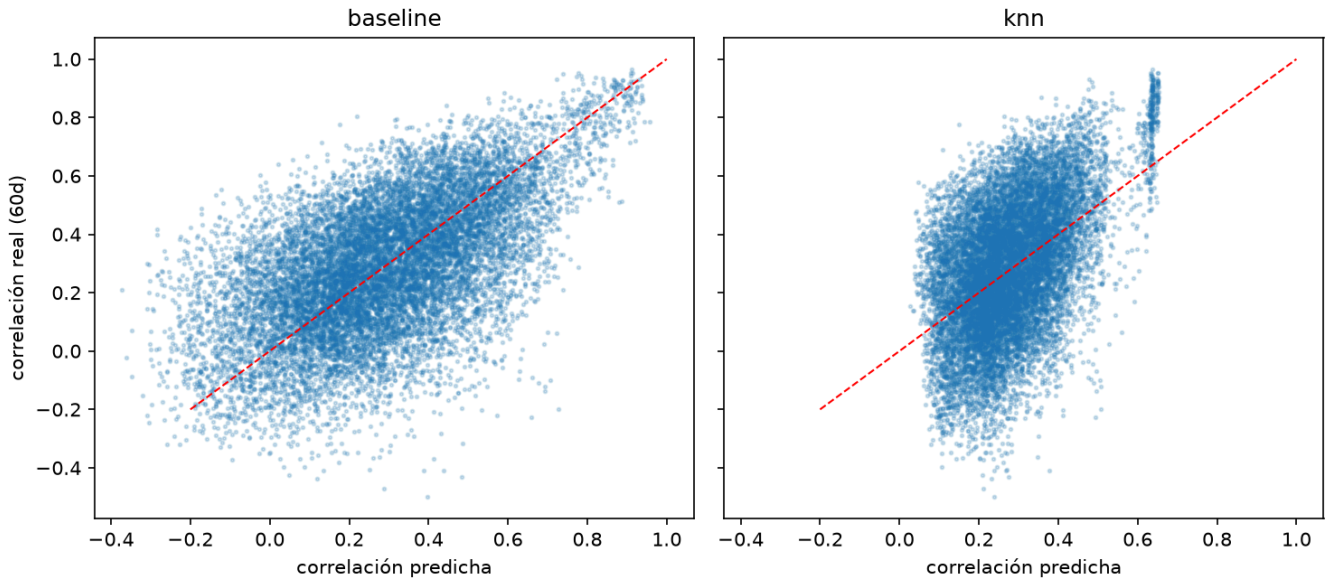
Resultados en test (14 355 filas):

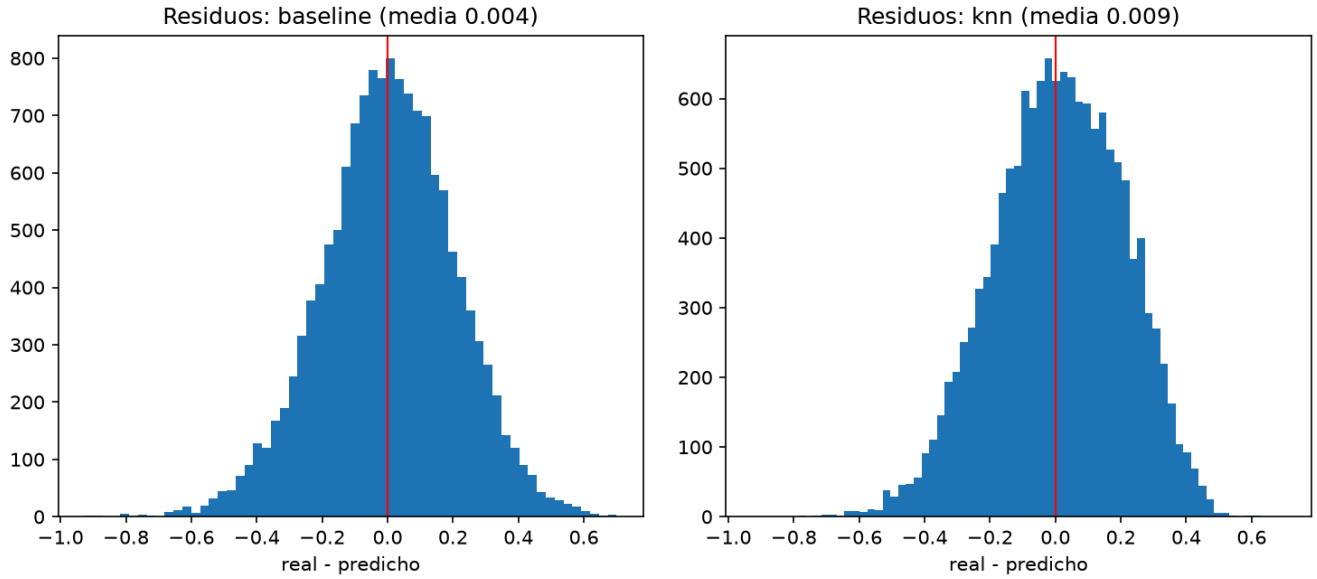
Modelo	MAE	vs. baseline	RMSE	R^2
Baseline (`corr_90d`)	0,1612	—	0,2042	0,211
Regresión lineal	**0,1564**	**−3,0 %**	0,1931	0,295
kNN (k = 1000)	0,1613	+0,1 %	0,1986	0,254
Árbol (profundidad 5, hoja mínima 5)	0,2011	+24,7 %	0,2480	−0,164

Elección por validación temporal (MAE de validación): kNN 0,2010; regresión lineal 0,2067; árbol 0,2185. **El modelo elegido por validación es el kNN**, y es el que se guarda en `data/modelo.joblib`.

Lectura de los resultados:

- **La mejora sobre el baseline es chica.** La regresión lineal baja el MAE un 3 % y sube el R^2 de 0,21 a 0,29. Hay señal, pero la correlación futura es difícil de predecir: gran parte es ruido de mercado, y `corr_90d` ya es un predictor fuerte.
- **El modelo elegido por validación (kNN) empata con el baseline en test.** La regresión lineal, que salió segunda en validación, es la que mejor anda en test. Elegir por validación es lo correcto (si no, se usaría el test para decidir), pero hay que reportar el resultado tal cual.
- **kNN eligió `k = 1000`, el extremo de la grilla.** Un `k` tan grande promedia casi todo y el modelo deja de distinguir pares parecidos; más flexibilidad no ayuda con estas features. No se probaron valores mayores.
- **El árbol es el peor**, peor que predecir la media (R^2 negativo). La causa probable es sobreajuste a features ruidosas; no se verificó comparando el error en train contra el de test.





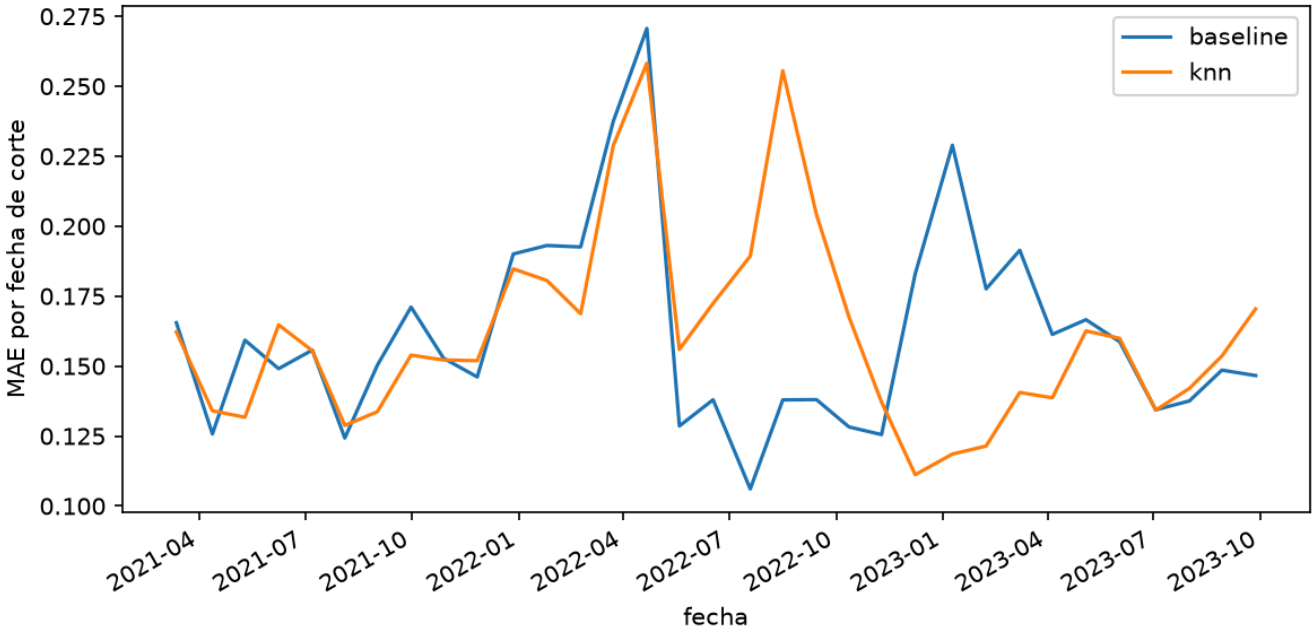
4.4 Análisis de errores

Un MAE promedio esconde dónde falla el modelo. Se segmentó el test (definición de *crisis*: `volatilidad_mercado` por encima del percentil 75 de train, umbral 0,0113, calculado sin mirar test).

MAE por segmento:

Segmento	Filas	Baseline	Regr. lineal	kNN	Árbol
Mercado en calma	8 700	0,156	**0,147**	0,153	0,206
Mercado en crisis	5 655	**0,169**	0,171	0,174	0,193
Distinto sector	13 431	0,163	**0,157**	0,162	0,203
Mismo sector	924	**0,138**	0,142	0,154	0,174

- **La mejora viene toda de los períodos de calma.** En crisis ningún modelo le gana al baseline.
- **El modelo subestima la crisis** (notebook `04-analisis-de-errores`, con la regresión lineal): en crisis la correlación real media es 0,38 y la predicha 0,28. Cuando el mercado se mueve en bloque las acciones se correlacionan más de lo que el modelo anticipa, porque todas las features miran el pasado y nada puede adelantar un shock que todavía no ocurrió.
- **Mismo sector:** son pocos pares (28 de 435, unas 900 filas de test) y ahí el baseline también gana.



---

5. Algoritmos de grafos

Todos los algoritmos se implementaron a mano sobre la **matriz de adyacencia**, y sus resultados se contrastan con `networkx` en los tests (`backend/tests/`).

5.1 Representación: el grafo de conflicto

Dada una fecha de corte, un umbral θ y una fuente de pesos, el grafo G = (V, E) es **simple, no dirigido y con pesos**:

- Nodos: las 30 acciones.
- Arista entre i y j si ρ(i,j) > θ (desigualdad estricta); peso de la arista = ρ(i,j).

La clase `GrafoConflicto` (`backend/app/grafos/construccion.py`) guarda dos matrices de 30 × 30: `pesos` (ρ de **todos** los pares) y `adyacencia` (0/1, sale de `pesos > θ`, simétrica,

diagonal en 0). Se guardan las dos porque cambiar θ solo cambia la adyacencia, y la evaluación necesita ρ de pares que no son arista. El grado de un nodo es la suma de su fila.

Con θ muy bajo el grafo se acerca al completo K_{30} (435 aristas) y hacen falta 30 canastas; con θ muy alto no hay aristas y alcanza con 1. Es el caso extremo que se ve al mover el selector de umbral.

5.2 Coloreo greedy por grado

Es el algoritmo que construye las canastas (`backend/app/grafos/coloreo.py`):

1. Se separa el grafo en **componentes conexas** (BFS) y se colorea cada una por separado. Como no comparten aristas, da el mismo resultado que colorear todo junto y muestra que $\chi(G) = \max \chi(G_i)$.
2. Dentro de cada componente se ordenan los vértices por **grado descendente** (desempate: menor índice, para que sea determinista).
3. A cada vértice se le asigna el **menor color que no usa ninguno de sus vecinos** ya coloreados.

Cada color es una canasta. El resultado es **siempre un coloreo propio** (ninguna arista con sus dos extremos del mismo color; el código lo valida), pero **puede usar más de χ(G) colores**: el greedy es una heurística.

Propiedades que se verifican en los tests: sin aristas usa 1 color; K_n usa n; un ciclo impar usa 3; usa a lo sumo Δ + 1 colores; nunca menos que el tamaño de la clique máxima (cota inferior de χ).

Hallazgo: el greedy por grado NO garantiza 2 colores en un bipartito. Contraejemplo: la corona de 8 nodos (bipartito, todos de grado 3) con los nodos intercalados: el greedy usa 4 colores y DSATUR usa 2. Por eso los tests de "bipartito ⇒ 2 colores" valen para casos concretos (caminos, ciclos pares, $K_{m,n}$, estrellas) y no como propiedad general.

5.3 Variantes: Welsh-Powell, DSATUR y coloreo con pesos

- **Welsh-Powell** es el mismo algoritmo: la versión clásica arma el color 0 completo, luego el 1, etc., pero da exactamente el mismo coloreo que el secuencial. Lo comprobamos en un test sobre 30 grafos al azar, en vez de implementarlo dos veces.
- **DSATUR** colorea siempre el nodo con más colores distintos entre sus vecinos (saturación) y es exacto en bipartitos. Sobre grafos al azar de 30 nodos usa casi siempre igual o menos colores que el greedy (por ejemplo, con p = 0,5: 8,39 contra 7,75 en promedio), y en bipartitos al azar el greedy usa más de 2 colores en el 13 % al 68 % de los casos mientras que DSATUR nunca lo hace (`documentacion/comparacion_coloreo.md`). Sobre los datos reales con `corr_90d` la diferencia es mínima (7,69 contra 7,64 canastas con θ = 0,5).

Mantenemos el greedy por grado como algoritmo principal (es el que pide la materia) y DSATUR como comparación.
- **Greedy con pesos (idea extra).** Misma regla de orden, pero entre los colores ya abiertos que ningún vecino usa, elige el de **menor suma de $\hat{\rho}$** con los nodos que ya tiene, es decir, la canasta donde el nodo queda menos correlacionado. Así el peso influye en el coloreo y no solo en qué aristas existen. Siempre da coloreo propio.

5.4 Contraste con el número cromático: polinomio cromático

El **polinomio cromático** $P(G,\lambda)$ cuenta de cuántas formas se puede colorear G con λ colores, y cumple $P(G,k) > 0 \Leftrightarrow k \geq \chi(G)$. Se calcula por **deleción-contracción** ($P(G) = P(G-e) - P(G/e)$), con caso base λ^n sin aristas y producto de los polinomios de las componentes conexas (`backend/app/grafos/cromatico.py`). Como el costo es exponencial, el código rechaza grafos de más de 20 aristas, y se aplica a subgrafos chicos, no al grafo de 30 acciones.

Sirve para **verificar el greedy**: $\chi(G)$ es el menor k con $P(G,k) > 0$. En 60 grafos al azar de 8 nodos ($p = 0{,}4$), el greedy por grado usó exactamente χ en 58 y más en 2, y nunca menos. Es la confirmación empírica de que es una heurística. Los tests contrastan además el polinomio con un conteo de fuerza bruta y con fórmulas conocidas (camino, ciclo, K_n).

5.5 Otras herramientas de la materia aplicadas al mismo grafo

(`backend/app/grafos/algoritmos.py`)

- **Test de bipartición** (BFS con 2 colores): si el grafo es bipartito, $\chi = 2$. Se usa también como test de corrección del greedy.
- **Complemento** $\bar{G}$: una canasta es un conjunto sin aristas entre sí, o sea una clique en el complemento. $\bar{G}$ une los pares poco correlacionados, que son los que conviene combinar. Es la segunda vista del frontend.
- **Nodos de quiebre:** v es nodo de quiebre si $k(G-v) > k(G)$ (se elimina el vértice y se cuentan las componentes). Señalan acciones que funcionan de puente entre grupos correlacionados.
- **Árbol recubridor mínimo (Prim)** sobre la distancia $d = \sqrt{2(1 - \rho)}$, que da una vista alternativa de cómo se agrupa el mercado. Usa la matriz completa de ρ, por lo que no depende de θ.

---

6. Integración: del modelo al grafo

La salida del modelo es $\hat{\rho}(i,j)$, la correlación futura predicha de cada par en una fecha de corte. El grafo se construye así:

1. Para una fecha de corte, el modelo predice $\hat{\rho}$ para los 435 pares.
2. Hay arista entre i y j si $\hat{\rho}(i,j) > \theta$, con peso $\hat{\rho}(i,j)$.
3. El coloreo greedy reparte las acciones en canastas, y χ (o la cantidad de colores usados) es el número de canastas.

Por qué es razonable. Una arista significa "es probable que estas dos acciones se muevan juntas en los próximos 60 días", y dos acciones unidas no pueden compartir canasta. El coloreo impone esa restricción para **todos** los pares a la vez, y la cantidad de canastas necesarias mide qué tan correlacionado está el mercado en esa fecha: cuantas más aristas, más canastas hacen falta.

Efecto de las crisis: se cumple con la correlación histórica pero no con el modelo. Esperábamos que en una crisis $\hat{\rho}$ subiera, aparecieran más aristas y hicieran falta más canastas. Medimos el número de canastas con $\theta = 0{,}5$ en las 111 fechas de corte:

Fuente	Canastas, enero 2020	Canastas, marzo-abril 2020	Correlación entre `volatilidad_mercado` y nº de canastas
`corr_90d`	7 a 8	**22 a 23**	**0,87**
Modelo (kNN, k = 1000)	4	3	−0,17

Con `corr_90d` el número de canastas es un buen indicador del estado del mercado: se triplica en la crisis de marzo de 2020. Con el modelo no: sus predicciones están muy suavizadas (kNN con k = 1000 promedia el target de muchísimos pares), así que casi no superan el umbral, el grafo queda ralo y el número de canastas no se mueve. Esto coincide con el análisis de errores (sección 4.4: el modelo subestima la correlación en crisis) y es una consecuencia concreta: **con la fuente `modelo` el grafo no refleja los cambios de régimen del mercado.** Esas fechas están en el período de train, así que ni siquiera es un problema de generalización. Es el punto más débil del sistema y conviene decirlo en la defensa.

El modelo importa. Si cambia $\hat{\rho}$, cambian las aristas, el coloreo y las canastas. Esa dependencia se ve en la aplicación al cambiar el umbral, la fecha o la fuente de correlaciones.

Fuentes de pesos intercambiables. El grafo se puede armar con tres fuentes, que devuelven el mismo tipo de objeto, así que el coloreo y la evaluación no saben de dónde vinieron los pesos: `modelo` (correlación predicha), `corr_90d` (la línea base) y `azar` (ρ uniforme en [−1, 1], el piso de comparación). Comparar fuentes es cambiar un parámetro.

Una limitación a tener presente. La garantía del coloreo es sobre la correlación **predicha**: ninguna canasta contiene un par con $\hat{\rho} > \theta$. Eso no garantiza que la correlación **real** de esos pares sea baja; cuánto se cumple en la práctica es exactamente lo que mide la sección 8.

---

7. ¿Por qué un grafo y no clustering?

Se podrían armar canastas sin grafos, por ejemplo con clustering jerárquico sobre la matriz de correlaciones. El aporte del grafo es el siguiente:

- **El clustering agrupa lo parecido; nosotros queremos agrupar lo distinto.** Un clustering sobre correlaciones junta las acciones que se mueven juntas, que es lo contrario de una canasta diversificada. Para obtener canastas diversificadas habría que tomar una acción de cada cluster, y eso ya no es una partición de las 30 acciones con una regla clara.
- **Garantía formal.** El coloreo asegura que **ningún par dentro de una canasta supera θ** (en términos de $\hat{\rho}$). El clustering no ofrece una cota sobre los pares de una misma canasta.
- **El número de canastas tiene una definición matemática.** $\chi(G)$ es el mínimo de canastas necesarias para cumplir esa garantía para un θ dado, y responde a "¿cuántas canastas hacen falta?". En el clustering el número de clusters es una elección arbitraria.
- **Parámetro interpretable.** θ se lee directamente ("no quiero pares con correlación mayor a 0,5"), y el resultado cambia de forma explicable al moverlo.
- **Herramientas adicionales sobre el mismo objeto:** polinomio cromático, complemento, nodos de quiebre y árbol recubridor mínimo responden preguntas que el clustering no.

Dos salvedades honestas: el greedy es una heurística y puede usar más canastas que el mínimo (χ), y **no implementamos ni comparamos empíricamente** un método de clustering, así que esta sección es un argumento y no un resultado medido.

Lo dejamos como argumento a propósito: el clustering no aplica ningún contenido de la materia Lógica y Teoría de Grafos, y este trabajo busca justamente integrar un modelo de Machine Learning con los algoritmos de grafos que vimos en la materia. Comparar con clustering queda como posible extensión (sección 11).

---

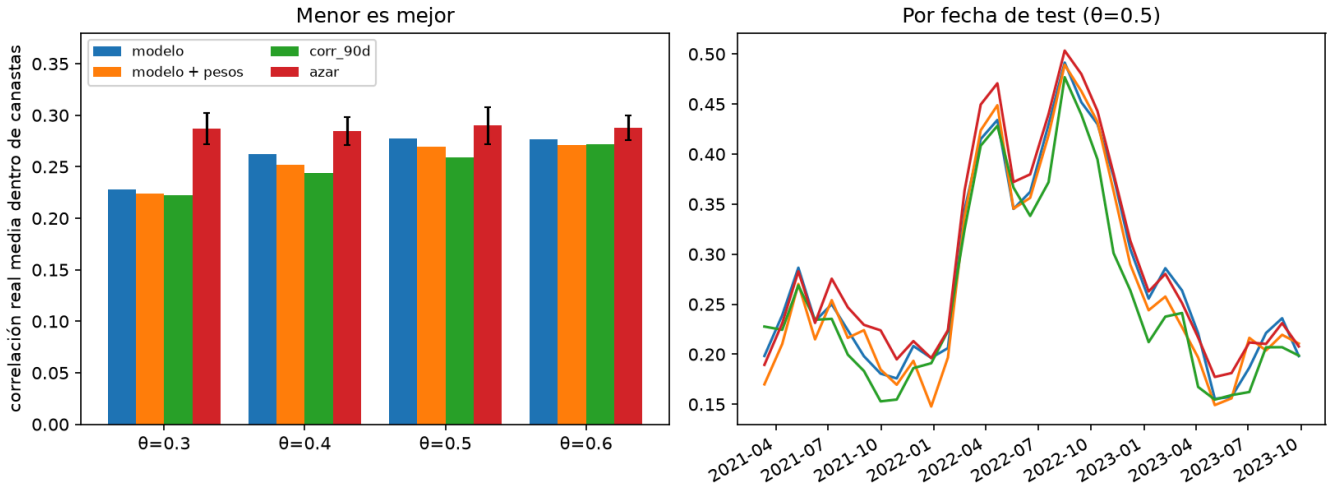
8. Evaluación de las canastas con la correlación real de test

Para cada una de las 33 fechas de test se arma el grafo con cada fuente de ρ, se colorea con el mismo greedy por grado y se mide, con la correlación **real** de los 60 días siguientes (`corr_futura_60d`), qué tan correlacionadas quedaron las acciones de una misma canasta. **Menos es mejor.** El azar usa 20 semillas por fecha (se promedian primero por fecha). El grafo

usa ρ̂ con datos hasta la fecha de corte y se evalúa con la correlación posterior, así que no hay fuga.

θ	Variante	Corr. real media en canastas	Pares con ρ real > θ	Canastas	Aristas
0,3	azar	0,287	0,481	6,7	152
0,3	corr_90d	0,223	0,347	12,1	202
0,3	modelo	0,228	0,365	9,8	172
0,3	modelo + pesos	0,224	0,352	10,0	172
0,4	azar	0,284	0,317	5,9	129
0,4	corr_90d	0,244	0,246	9,5	133
0,4	modelo	0,262	0,277	5,7	60
0,4	modelo + pesos	0,252	0,266	5,8	60
0,5	azar	0,290	0,187	5,2	108
0,5	corr_90d	0,259	0,129	6,9	74
0,5	modelo	0,278	0,157	3,3	15
0,5	modelo + pesos	0,270	0,158	3,3	15
0,6	azar	0,288	0,087	4,7	86
0,6	corr_90d	0,272	0,059	4,7	33
0,6	modelo	0,276	0,064	3,0	9
0,6	modelo + pesos	0,271	0,063	3,0	9

(Desvío del azar entre semillas, en la correlación media: 0,012 a 0,018 según θ.)



Lectura:

- **Todo le gana al azar** en todos los umbrales: el método funciona. Las canastas armadas con correlaciones (predichas o históricas) quedan menos correlacionadas que las armadas con ρ aleatorio.
- **El modelo no le gana a `corr_90d`.** Con θ = 0,3 empatan (0,228 contra 0,223) y con umbrales más altos `corr_90d` queda mejor (0,259 contra 0,278 con θ = 0,5). Es coherente con el R^2 chico del modelo sobre el baseline.
- **La comparación a igual θ no es del todo justa.** El modelo predice correlaciones más comprimidas hacia la media, así que supera el umbral menos veces: genera grafos más ralos (15 aristas contra 74 con θ = 0,5) y usa menos canastas (3,3 contra 6,9). Menos canastas obliga a juntar más acciones, y eso sube la correlación interna. Comparando con una cantidad parecida de canastas, el modelo con θ = 0,4 (5,7 canastas, 0,262) y `corr_90d` con θ = 0,5 (6,9 canastas, 0,259) quedan casi empatados. **Pendiente:** comparar a igual cantidad de canastas o elegir θ por fuente.
- **El coloreo con pesos mejora levemente** la correlación real frente al greedy común del modelo (por ejemplo 0,270 contra 0,278 con θ = 0,5) sin usar más canastas.
- **Las 33 fechas de test no son independientes:** están separadas por 20 días y el target mira 60, así que fechas vecinas comparten parte del período. No conviene sacar significancia estadística de ellas.

---

9. Aplicación web

9.1 Arquitectura

- **Backend (FastAPI)**: contiene toda la implementación (preparación de datos, modelo, algoritmos de grafos) y la expone por una API. Carga el modelo y el dataset una sola vez,

valida los parámetros (422) y responde 404 ante fechas inexistentes. Los tests de la API están en `backend/tests/test_api.py`.

- **Frontend (React + TypeScript + Vite)**: consume la API; usa Tailwind y componentes de shadcn/ui. El slider de umbral consulta con *debounce* de 300 ms y cancela los pedidos anteriores.

Endpoints principales:

Endpoint	Qué devuelve
`GET /fechas`	Fechas de corte disponibles
`GET /grafo?fecha&umbral&fuente`	Nodos con su canasta y grado, aristas con peso, y número de colores
`GET /canastas?fecha&umbral&fuente`	Canastas con sus acciones y correlación real posterior promedio
`GET /chi-por-umbral?fecha&fuente`	Canastas y aristas para una grilla de umbrales
`GET /comparacion?umbral[&fecha]`	Modelo vs. `corr_90d` vs. azar (sección 8)
`GET /complemento`, `/prim`, `/nodos-quiebre`, `/polinomio`	Herramientas de grafos de la sección 5
`GET /metricas`	Métricas del modelo (sección 4)

9.2 Pantallas

La pantalla tiene una barra superior (fecha de corte, umbral θ, fuente de correlaciones), una lista de acciones agrupadas por canasta, un panel central con pestañas y un panel derecho con las canastas.

- **Precio:** serie de la acción elegida, con la fecha de corte y las 60 ruedas a predecir.
- **Canastas según θ:** número de canastas para cada umbral (la curva que va de 30 a 1).
- **Grafo:** las acciones como nodos pintados por canasta, con vista de conflicto y vista de complemento.
- **Modelo vs. histórica:** la comparación de la sección 8.
- **Métricas del modelo:** las tablas de la sección 4.
- **Panel de canastas:** acciones de cada canasta y su correlación real posterior.

9.3 Capturas

Todas las capturas usan la fecha de corte 2023-09-27 (la última de test).

Figura 1. Pantalla completa (fuente Modelo, θ = 0,5, pestaña Precio). A la izquierda, las 30 acciones agrupadas por canasta con su precio y variación mensual; en el centro, el precio de la acción elegida con la fecha de corte y las 60 ruedas a predecir; a la derecha, las 3 canastas que resultan, con su correlación real posterior.

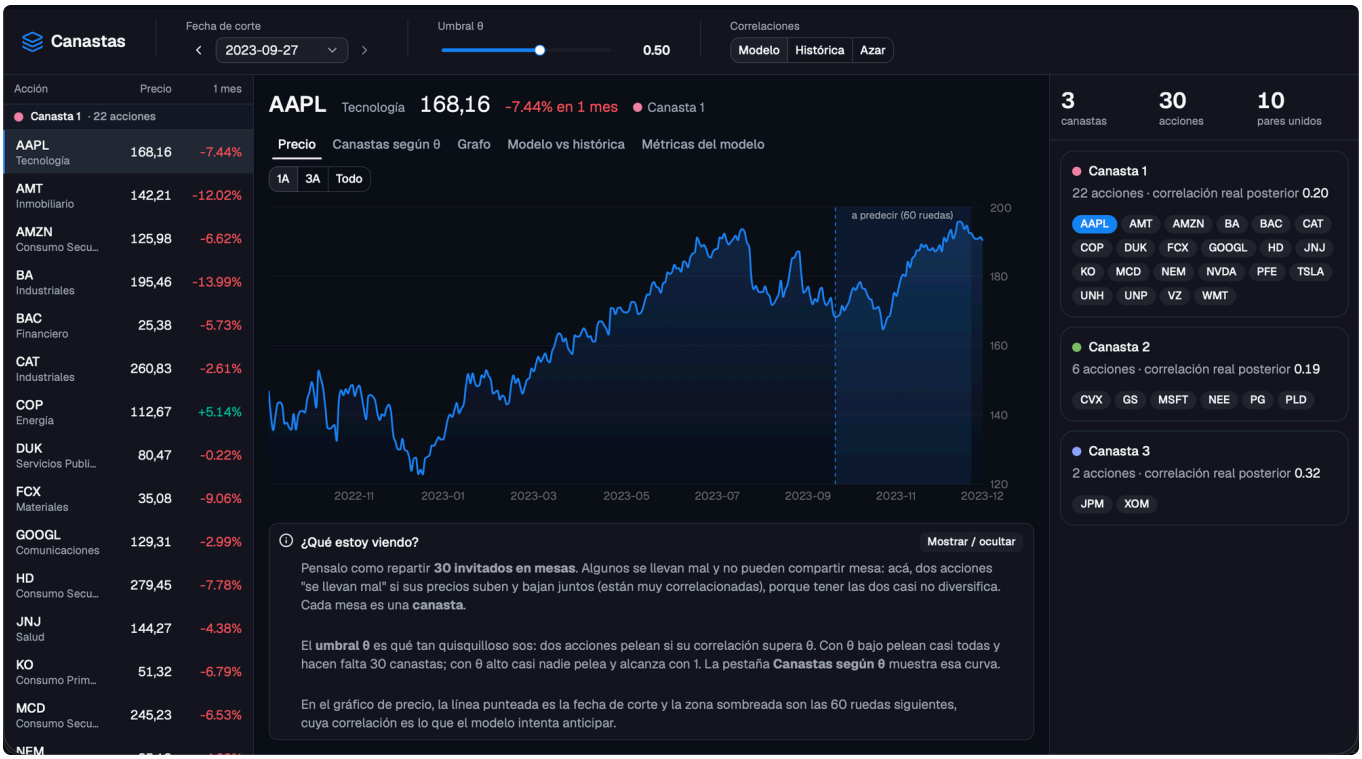


Figura 2. Grafo coloreado con θ = 0,5, armado con la fuente Modelo (arriba, 10 aristas) y con la fuente Histórica (abajo, 15 aristas). Cada nodo es una acción, el color indica la canasta y una arista une a las acciones cuya correlación supera θ. En ambos casos hacen falta 3 canastas, pero el grafo del modelo es más ralo.

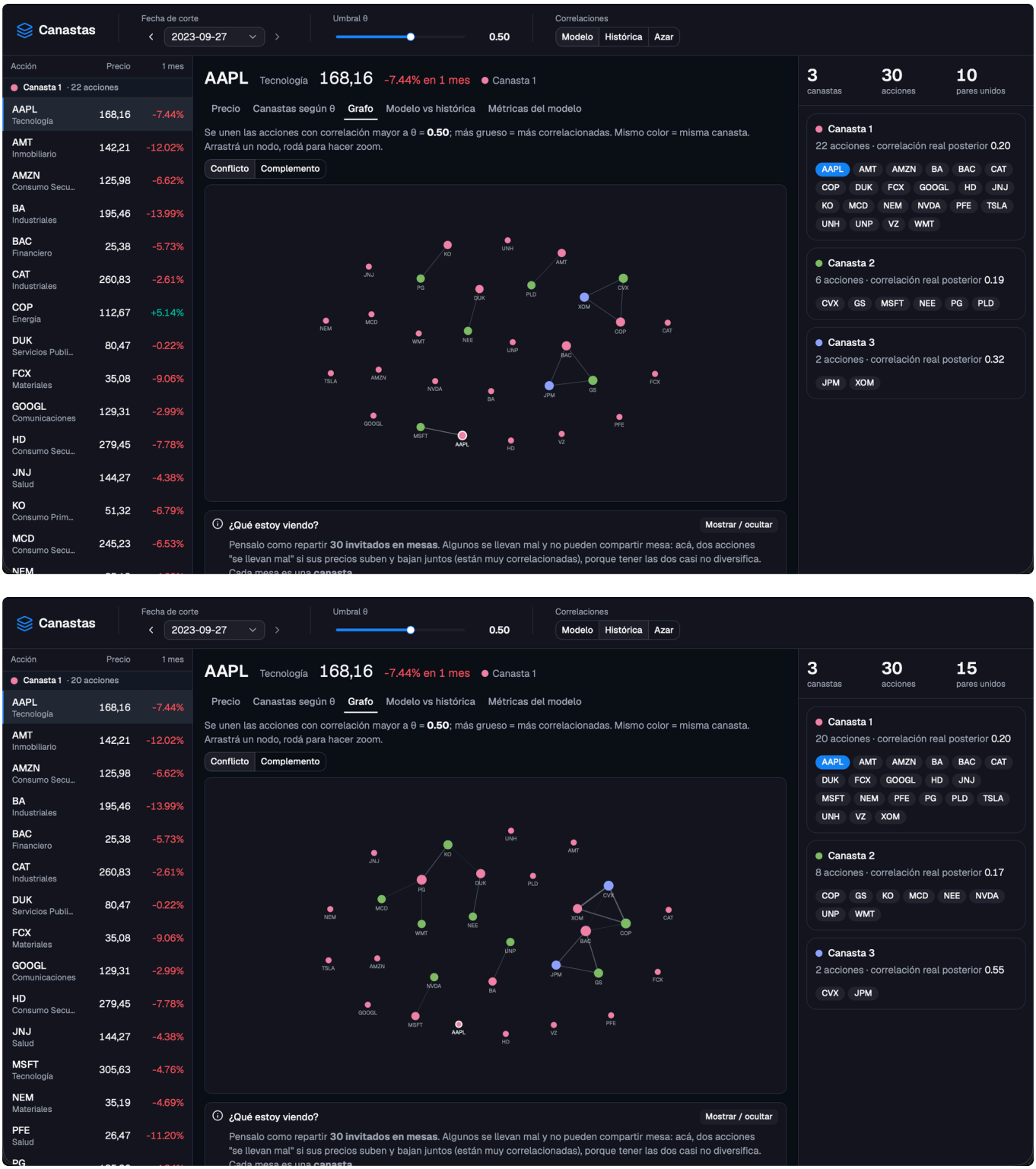


Figura 3. Los dos extremos del umbral. Con θ = −0,2 todos los pares superan el umbral: el grafo es el completo K_{30} (435 aristas) y hacen falta 30 canastas, una por acción. Con θ = 1 ningún par lo supera: no hay aristas y alcanza con una sola canasta.

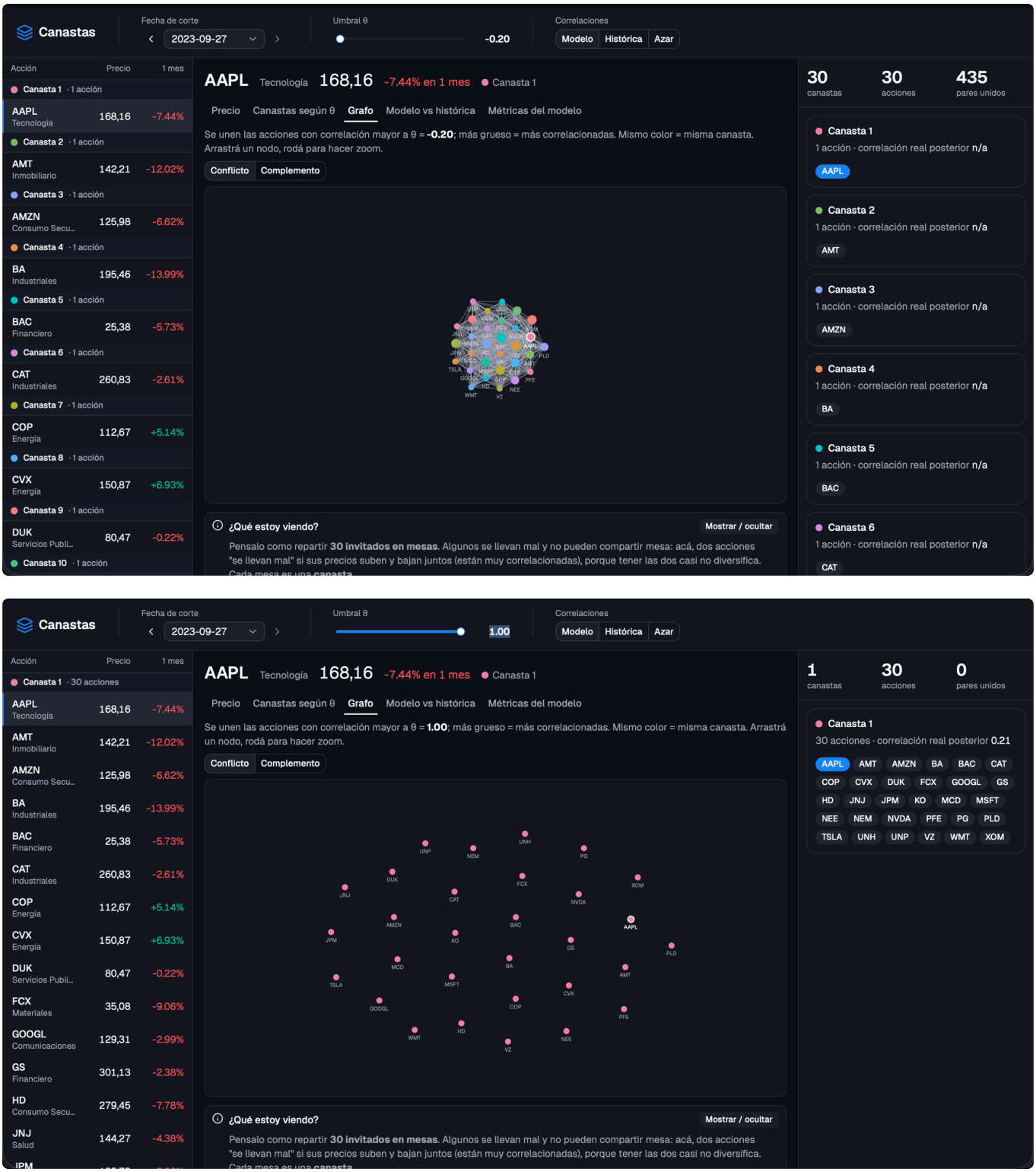
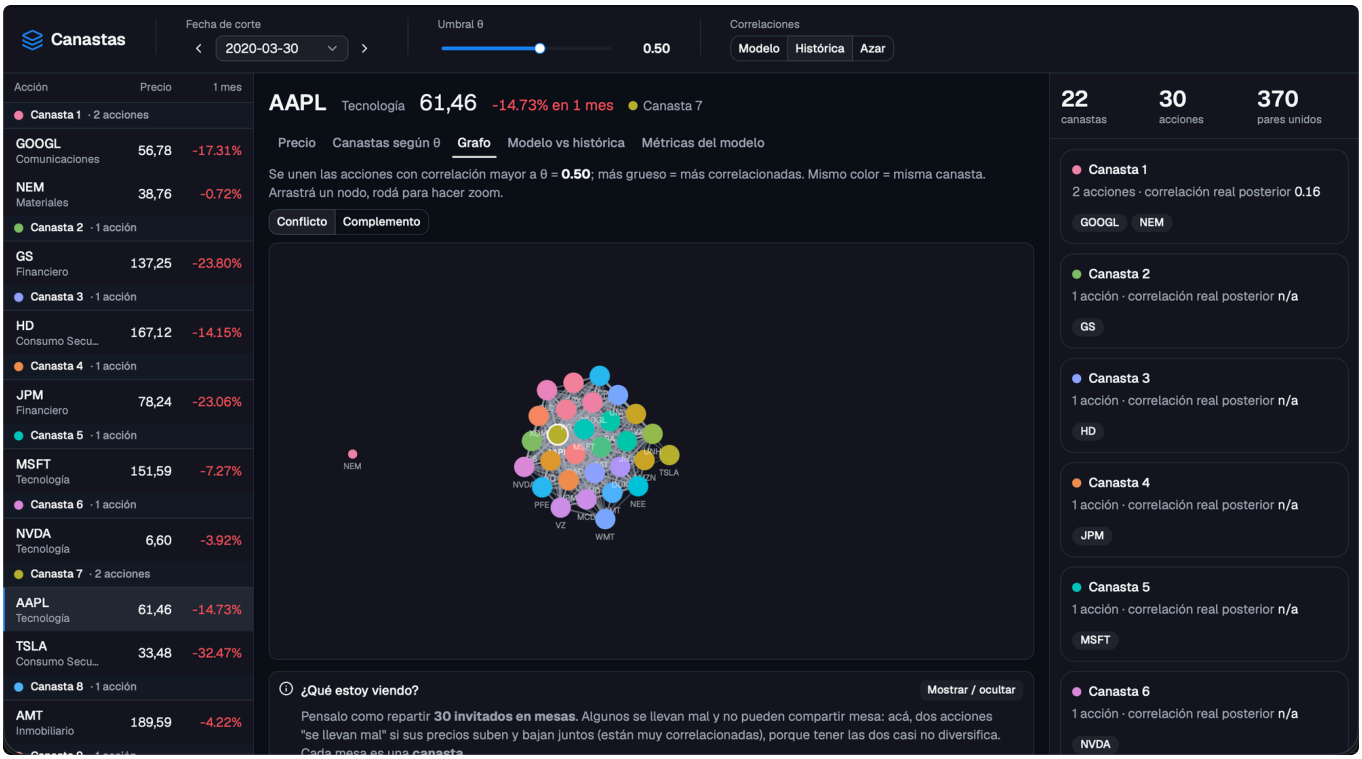


Figura 4. La crisis de marzo de 2020 (fecha de corte 2020-03-30, θ = 0,5, pestaña Grafo). Con la fuente **Histórica** el grafo es denso (370 aristas) y hacen falta **22 canastas**; con la fuente **Modelo** es ralo (21 aristas) y alcanzan **3 canastas**. Es la diferencia de la sección 6: el grafo armado con la correlación histórica refleja el salto de correlaciones de una crisis, y el armado con el modelo no.



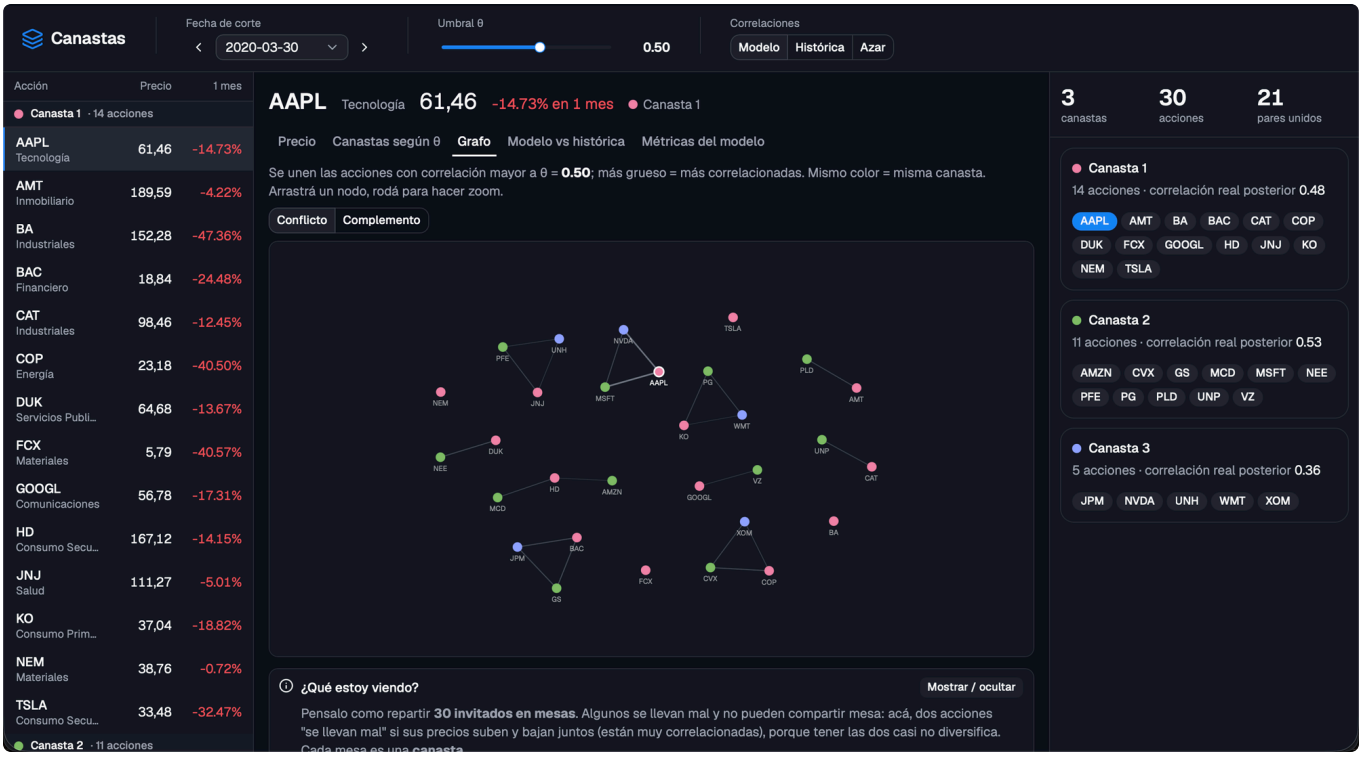


Figura 5. Canastas necesarias según el umbral (fecha de corte 2023-09-27, fuente Histórica, pestaña "Canastas según θ"). A la izquierda, con θ bajo, casi todos los pares superan el umbral y hacen falta cerca de 30 canastas; a la derecha, con θ alto, alcanza con una. La línea punteada marca el θ elegido (0,5), que en esta fecha requiere 3 canastas. Con la fuente Modelo la curva es más brusca: se queda en 30 canastas hasta θ ≈ 0 y baja de golpe.



Figura 6. Comparación de las canastas del modelo, de la correlación histórica y del azar (pestaña "Modelo vs. histórica", promedio de las 33 fechas de test, detalle con θ = 0,5). Las barras muestran la correlación **real** media dentro de las canastas para cada umbral (menos es mejor), y la tabla da el detalle con θ = 0,5. Todos los métodos superan al azar. La histórica queda mejor que el modelo en correlación media (0,259 contra 0,278), pero usa más del doble de canastas (6,9 contra 3,3) y de aristas (74 contra 15), por eso la pestaña incluye el aviso sobre comparar con el mismo θ.

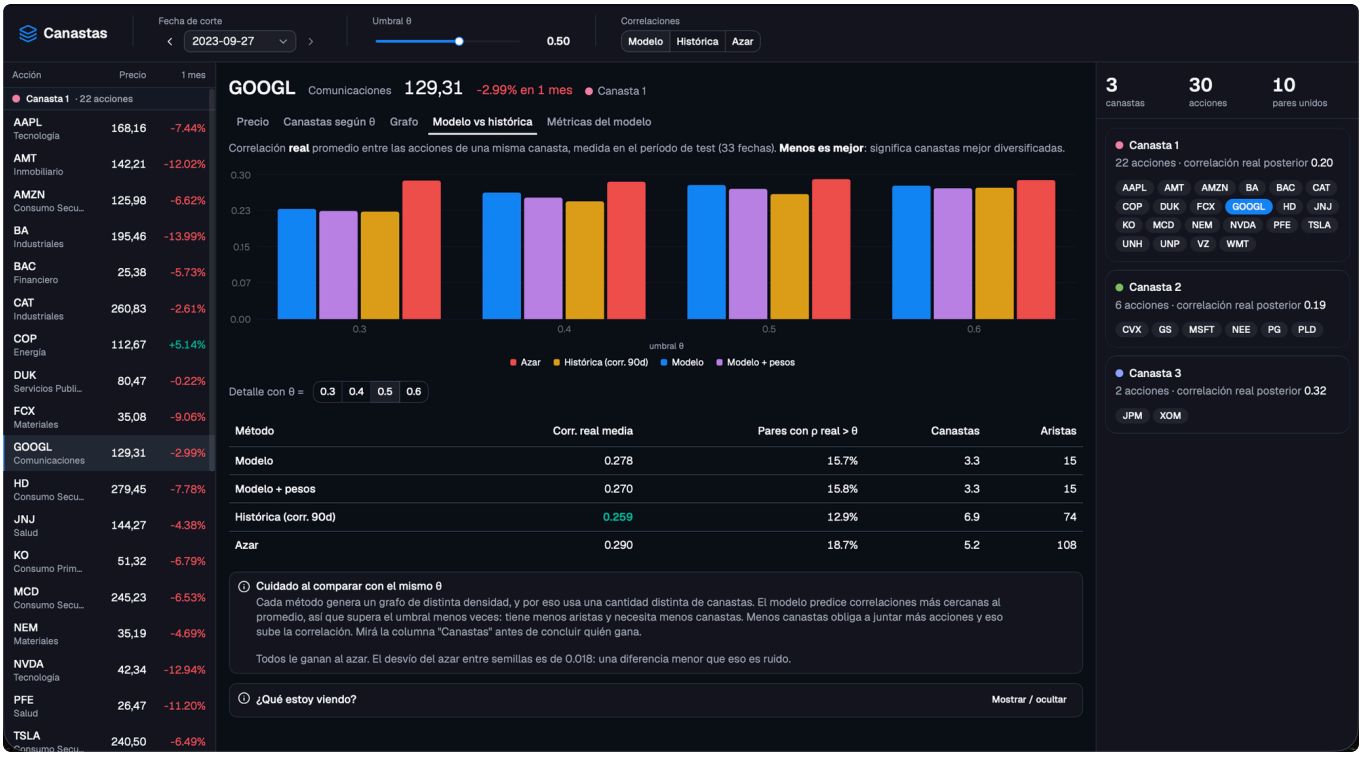


Figura 7. Métricas del modelo (pestaña "Métricas del modelo"). La primera tabla compara en test los tres modelos con el baseline (MAE, mejora respecto del baseline, RMSE y R^2) y marca el elegido por validación; la segunda da el MAE por segmento. En el total, solo la regresión lineal mejora al baseline (3 %). Por segmento, la lineal y el kNN lo mejoran en calma y en pares de distinto sector (que son la gran mayoría), mientras que en crisis y en pares del mismo sector gana el baseline.

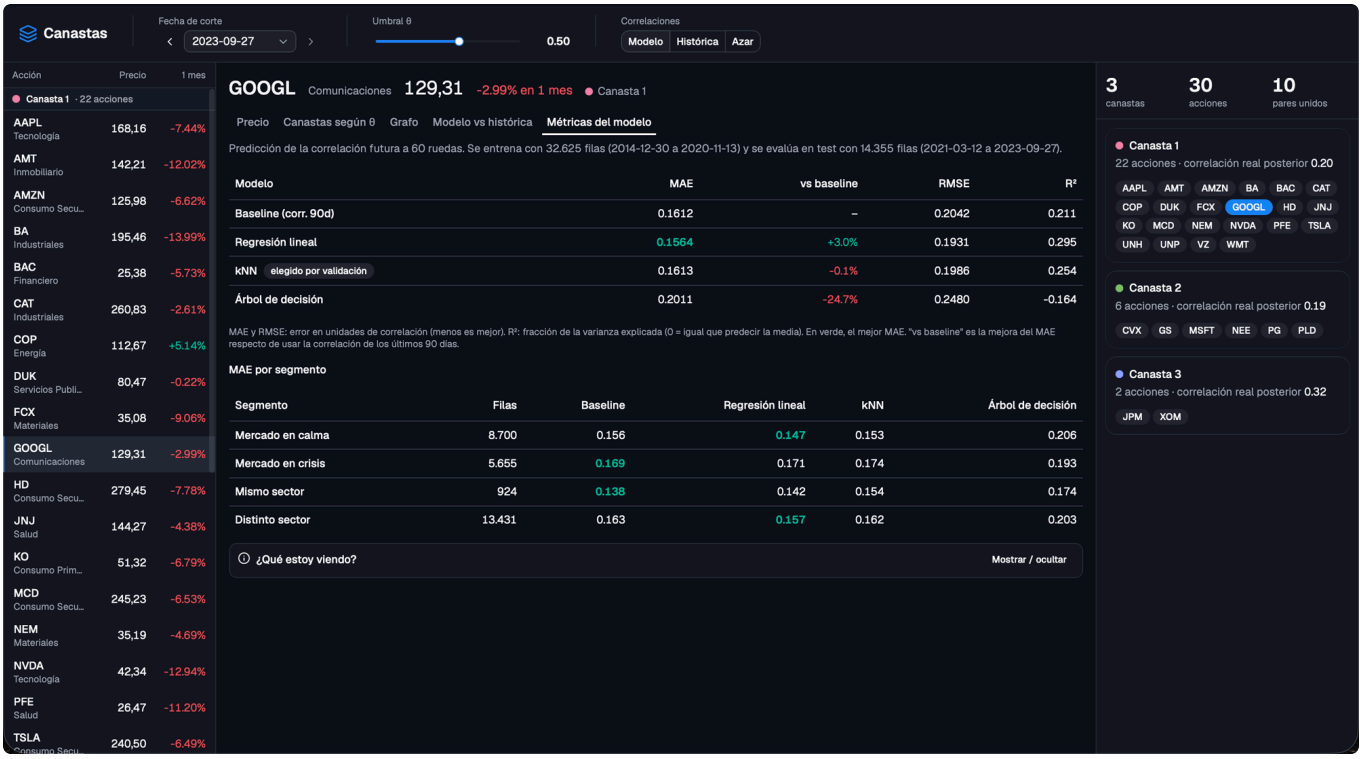


Figura 8. Árbol mínimo y nodos de quiebre (fecha de corte 2023-09-27, θ = 0,35, pestaña Grafo). Los botones "Árbol mínimo" y "Nodos de quiebre" superponen sobre el grafo las uniones del árbol recubridor mínimo de Prim, que conecta las 30 acciones con 29 uniones de distancia total 28,03 y no depende de θ, y marcan los nodos de quiebre del grafo de conflicto (AMT y COP: sacarlos deja más componentes).

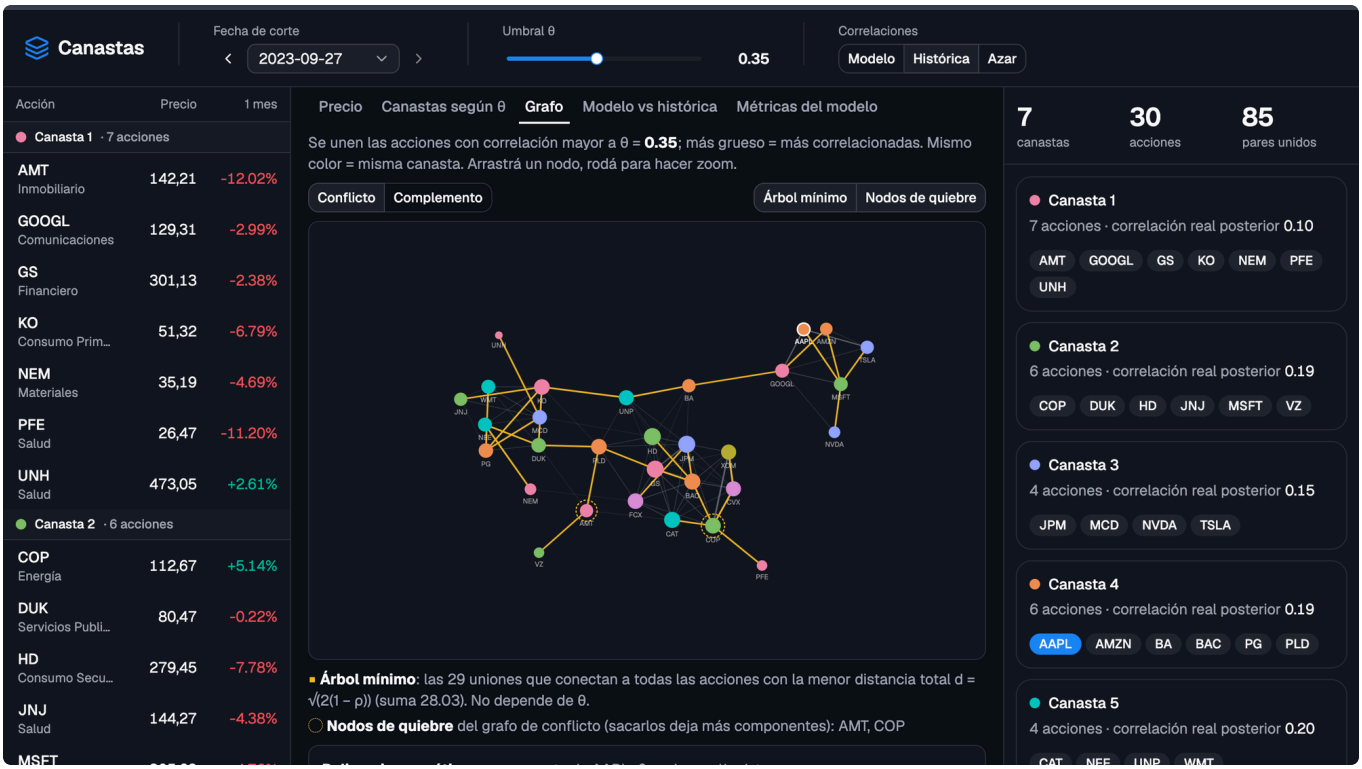


Figura 9. Vista de complemento (acción seleccionada BAC, fecha 2023-09-27, θ = 0,43). Se unen las acciones que sí pueden ir juntas (correlación ≤ θ), con más grosor cuanto menos correlacionadas están. Cada canasta queda como un grupo donde todas se unen con todas, es decir, una clique del complemento.

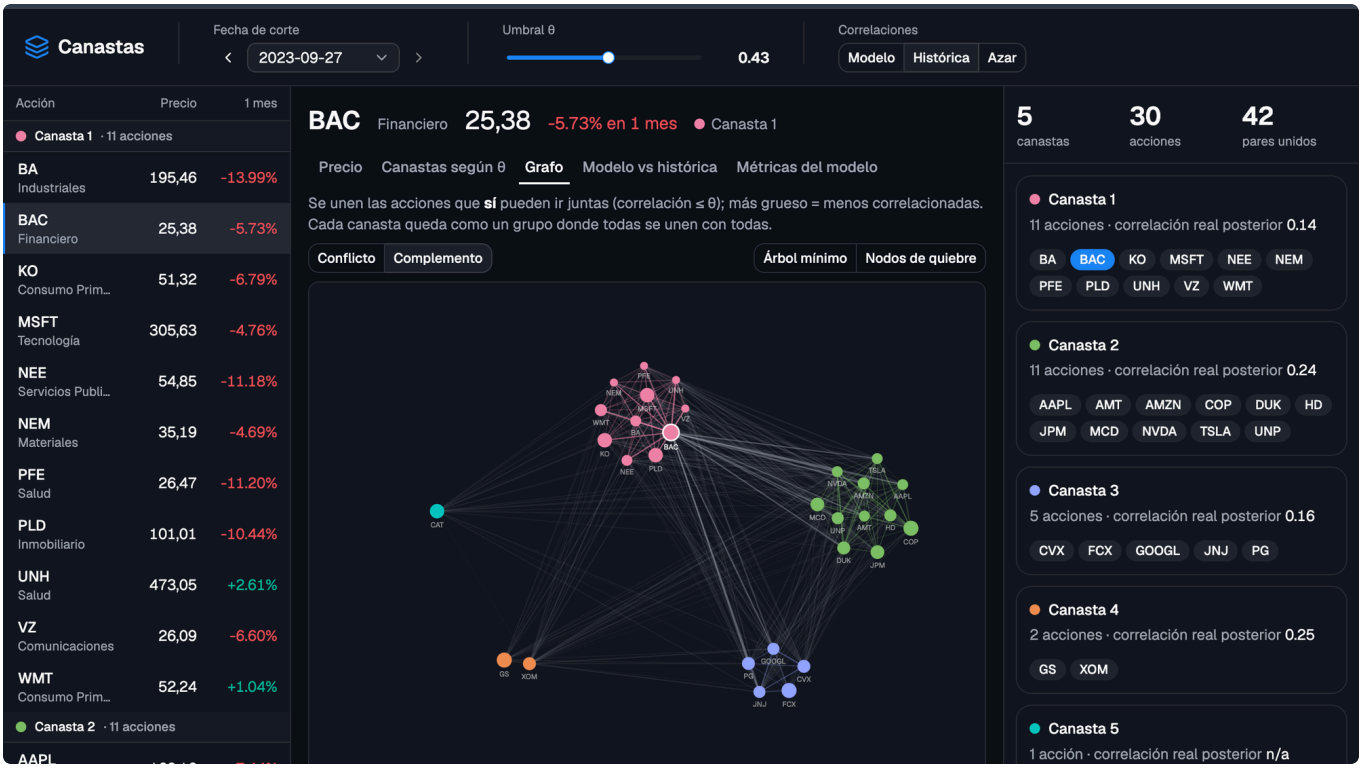


Figura 10. Polinomio cromático (fecha 2023-09-27, θ = 0,43). El panel calcula P(G, λ) para la componente conexa de la acción elegida (MSFT: 6 acciones y 6 aristas): $P(G, \lambda) = \lambda^6 - 6\lambda^5 +$

$14\lambda^4 - 16\lambda^3 + 9\lambda^2 - 2\lambda$, de donde $\chi(G) = 3$ (el menor k con $P(G, k) > 0$). El greedy también usó 3 colores en esa componente, así que coincide con el óptimo (sección 5.4).

Polinomio cromático componente de MSFT: 6 acciones, 6 aristas

$$P(G, \lambda) = \lambda^6 - 6\lambda^5 + 14\lambda^4 - 16\lambda^3 + 9\lambda^2 - 2\lambda$$

$\chi(G) =$ **3** (el menor k con $P(G, k) > 0$). El greedy usó **3** colores en esta componente: coincide con el óptimo.

---

10. Uso de inteligencia artificial

Usamos IA en prácticamente todo el trayecto del trabajo, desde el planteo del problema hasta la mejora de la redacción de este informe. La herramienta principal fue Claude (de Anthropic), sobre todo a través de Claude Code; también usamos Gemini (de Google) para investigar la problemática.

10.1 Herramientas

- **Claude Code** (la herramienta de línea de comandos de Anthropic), corriendo dentro del repositorio. Leía y editaba archivos, ejecutaba `pytest` y los scripts, y hacía los commits cuando se lo pedíamos.
- **Un archivo** `CLAUDE.md` con las reglas del dominio (sin fuga de información, partición temporal, algoritmos de grafos implementados a mano, librerías permitidas), para que respetara las restricciones del trabajo en cada sesión. Está en `.gitignore` (decisión 9), por eso no figura en el repositorio.
- **La memoria de Claude Code**, donde quedó guardada la regla de usar solo numpy, pandas, matplotlib y scikit-learn.
- **Gemini** (Google), para investigar el tema base y qué proyectos parecidos existían en la red, y tener una idea antes de trabajarlo.

10.2 Metodología de trabajo

El trabajo con IA fue un ida y vuelta. Empezamos investigando con Gemini el tema base y qué proyectos parecidos había en la red, para tener una idea de por dónde ir. Con eso trabajamos la idea con Claude hasta llegar a una propuesta bien planteada (`PROPUESTA.md`). Después implementamos con Claude Code: pegábamos cada ítem de la lista de tareas como prompt, le pedíamos que nos hiciera preguntas antes de implementar, revisábamos cada cambio, corríamos los tests y hacíamos commits chicos. Cada decisión de diseño quedó documentada con su fecha en `documentacion/decisiones.md` .

10.3 Dónde se usó

Etapa	Qué se hizo con IA	Dónde queda registrado
Planteo del problema	Investigación del tema base y de proyectos existentes (Gemini). Después, trabajar la idea con Claude hasta llegar a una propuesta bien planteada	`PROPUESTA.md` (commits del 20/10)
Descarga y dataset por pares	Script de `yfinance`, features rodantes y el test anti-fuga	Decisiones 12 a 14
Modelos	Partición temporal, validación con gap y comparación contra el baseline `corr_90d`	Decisiones 19 a 21
Grafo de conflicto	Clase `GrafoConflicto` con matriz de adyacencia y fuentes de pesos intercambiables	Decisión 22
Algoritmos de grafos	Coloreo greedy por grado, DSATUR, bipartito, complemento, nodos de quiebre, Prim y polinomio cromático, con sus tests	Decisiones 23 a 25
Evaluación de canastas	Modelo contra `corr_90d` y azar con varias semillas, y coloreo con pesos	Decisión 26
API	Endpoints de FastAPI y sus tests	`backend/app/main.py`, `backend/tests/test_api.py`
Frontend	Estructura con Tailwind y shadcn/ui, diseño estilo plataforma de trading, selectores de umbral y fecha con *debounce*, lista de acciones por canasta, gráficos, pestañas de comparación y métricas, grafo animado estilo Obsidian, árbol mínimo, nodos de quiebre y polinomio, y la explicación en lenguaje llano dentro de la web	`frontend/src/`, decisión 28
Notebooks	Los cuatro notebooks, escritos con el estilo de las clases de Machine Learning	`notebooks/`
Análisis y verificaciones	Comprobar el gap con datos reales, medir cómo cambia el número de canastas en la crisis de 2020 y	Secciones 4.2 y 6, `README.md`

Etapa	Qué se hizo con IA	Dónde queda registrado
	probar el README en un clon limpio del repositorio	
Documentación	`decisiones.md`, `preguntas-informe.md`, `ACCIONES.md`, el README, el borrador del informe y su redacción	`documentacion/`, `README.md`, `informe/`
Planificación	Armar los issues, los hitos con sus fechas y el reparto de tareas entre los dos	Issues y milestones del repositorio
Gestión del repositorio	Mensajes de commit, comentarios al cerrar issues y el cambio de nombre del repositorio	Historial de commits e issues

Una aclaración sobre los commits: algunos llevan la línea de coautoría de Claude y otros no. Por eso la ausencia de esa línea en un commit no significa que no se haya usado IA; el registro completo es esta sección.

10.4 Qué decidimos y hicimos nosotros

Lo siguiente lo decidimos o hicimos nosotros, sin delegarlo:

- **El tema del proyecto**, que después trabajamos con Claude hasta llegar a una propuesta bien planteada (`PROPUESTA.md` , 20/10), y la **aprobación del tema propio** por parte de los docentes.
- **Las 30 acciones y los sectores.** Elegimos las acciones por ser conocidas y editamos `sectores.csv` a mano.
- `SPY` **como referencia del mercado** (el cambio de `^GSPC` a `SPY` , decisión 11), porque es conocido en el ámbito financiero, tiene mucho renombre y es donde invierte la mayoría de la gente.
- **La investigación sobre Yahoo Finance:** averiguamos nosotros si tenía una API para obtener los datos de las acciones.
- **La ventana de 90 días** para la volatilidad y el beta, que eligió Manuel (decisión 15).
- **Qué features implementar y en qué orden.** Pegábamos cada ítem de la lista de tareas como prompt.
- **Las librerías permitidas** (numpy, pandas, matplotlib, scikit-learn). Por eso se reemplazó `networkx` en `app/` por `matplotlib` (decisión 27).
- **Cómo trabajar entre los dos.** Como ya trabajamos mucho juntos, definimos pautas para facilitar la cooperación: documentar cada decisión y pedirle a la IA cosas que ya conocemos, por ejemplo la estética de grafo de Obsidian.

- **Reducir lo desconocido y movernos dentro del entorno que nos dieron los profes.** El conocimiento base de Machine Learning lo traíamos de la escuela. Hicimos notebooks como los del curso de ML para que se comprenda mejor el código, y una interfaz parecida a los ejercicios de coloreo que resolvimos en clase, pero más linda.
- **Tener las suites de tests necesarias** para detectar errores de código.
- **La revisión de cada cambio y los commits chicos.** En la API, el frontend y el informe, los commits y los `push` los hicimos nosotros, y la IA consultaba antes de cerrar issues, borrar archivos o renombrar el repositorio. En la parte de grafos, Claude Code hacía los commits cuando se lo pedíamos.
- **Las capturas del frontend** con la aplicación funcionando, y el **contraste de los resultados** informados con los archivos que generan los scripts (`data/metricas.json` , `data/evaluacion_canastas.csv`).

10.5 Ejemplos concretos de prompts útiles

1. **Pasar el enunciado de la tarea tal cual y pedir preguntas antes de hacer nada:** *"Tenemos que implementar esto ahora, haceme todas las preguntas que necesites, explicame qué hiciste: [lista de la tarea de coloreo greedy]"*. Evitó suposiciones sobre el diseño.
2. **Pedir una verificación del trabajo contra lo pedido:** *"¿Cumpliste con todo lo que te pedí en el prompt anterior? Haceme una lista de cómo lo resolviste"*. Sirvió para detectar faltantes.
3. **Pedir que compare con alternativas:** *"Armá la tabla comparativa, quiero saber más sobre el test de bipartito, ¿qué significa que no se cumpla en general?"*. De ahí salió `comparacion_coloreo.md` (greedy contra DSATUR).
4. **Pedir una explicación antes de aceptar el código:** *"Explicame en qué consiste la implementación tuya de esta feature"* y *"¿Qué es el d de Prim?"*. Necesario para poder defender el trabajo oralmente.
5. **Preguntar por el comportamiento esperado:** *"¿Qué tendría que pasar cuando tengo un theta muy bajo o muy alto? ¿Y cuando tengo una fecha de crisis?"*. De ahí salió un hallazgo para el informe (sección 6).
6. **Fijar restricciones:** *"Usemos siempre numpy, pandas y matplotlib"* y *"Hacé la estética del grafo oscilante como los grafos de bóvedas en Obsidian"*.
7. **Registrar mientras se trabaja:** *"Anotá lo que hiciste"* y *"anotalo en preguntas-informe"*. Así las decisiones quedaron documentadas con la fecha.
8. **Definir el diseño de la pantalla:** *"quiero que la web tenga una vista como si fuera de trading view, además de que use componentes de shadcn"*.
9. **Pedir que la explicación esté en la aplicación:** ante un gráfico que no se entendía, *"explicá eso dentro de la web"*. Llevó a explicar el umbral con la analogía de repartir invitados en mesas.

10. **Auditar el trabajo contra la consigna:** *"revisá si cumple con este enunciado en donde están las pautas"*. Sirvió para detectar lo que faltaba (la sección de IA y las capturas).

10.6 Casos en que la IA se equivocó y cómo nos dimos cuenta

1. **Test con valor esperado equivocado (5/11).** Claude armó los tests de la construcción del grafo y uno falló: esperaba una arista A-C con ρ = 0,5 y θ = 0,5, pero la arista exige ρ > θ estricto, así que no existe. Lo detectó `pytest` al correrlo, Claude reconoció que el error estaba en el test y no en el código, y corrigió lo esperado. Se anotó la regla de la desigualdad estricta en la decisión 22.
2. **Se implementó sin la lista completa (6/11).** Claude implementó una feature a partir de un pedido incompleto (nos habíamos olvidado de pegar la lista). Lo notamos al leer el resumen y pedimos *"borrá los cambios que hiciste después del commit"*. Fue un error nuestro de prompt, pero muestra que Claude avanza con lo que tiene en vez de frenar. Después pegamos siempre la lista completa.
3. **Tomó un issue que no nos tocaba.** Al pedirle que hiciera los issues vencidos, empezó por uno asignado al otro integrante (el coloreo greedy). Lo notamos al ver que no era nuestro y se descartó ese trabajo.
4. **Interfaces iniciales difíciles de entender.** Las primeras interfaces que generó la IA eran confusas para nosotros, y hubo que corregirlas hasta que fueran legibles, aunque quedaran más pesadas para la computadora. Lo notamos al probarlas a mano y tratar de explicarlas, algo que tenemos que poder hacer en la defensa oral.
5. **Un layout que tapaba las tablas.** Las pestañas de comparación y de métricas se montaban sobre el panel de explicación y ocultaban filas. Lo detectamos en las capturas y se corrigió.
6. **Una frase del informe afirmaba que en una crisis el número de canastas sube.** Al medirlo en las 111 fechas de corte, se cumplía con la correlación histórica (de 7 a 22 canastas en marzo de 2020) pero no con el modelo (se queda en 3). Se corrigió el texto y pasó a ser un hallazgo del informe (sección 6).
7. **Un porcentaje mal leído en `decisiones.md`.** Decía que el greedy usa más de 2 colores en bipartitos "27 % a 68 %"; al armar el informe se recalculó con la tabla y era 13 % a 68 %.
8. **Decir que el gap de 3 cortes "hacía falta".** Al comprobarlo con datos reales (sección 4.2) se vio que el mínimo es 2 y que 3 es una elección conservadora. Se corrigió la redacción.
9. **Documentación en el lugar equivocado (8/11).** Al registrar la estética del grafo, Claude escribió el texto fuera de la decisión que correspondía. Lo vio al releer el archivo y lo movió. Lección: revisar los `.md` igual que el código.
10. **Un archivo de prueba que no corría (5/11).** Nuestro script `probar_grafo.py` fallaba. Claude creó una versión corregida y dejó el original, porque el sistema bloqueó el borrado

de un archivo nuestro, así que quedaron dos copias. Se limpió después: hoy ya no queda ninguna en el repositorio.

11. **README incompleto (8/11).** Al levantar el frontend y el backend siguiendo solo el README, vimos que faltaba el paso de entrenar el modelo. Sin ese paso la aplicación no iba a funcionar cuando la probaran los profes. Lo corregimos en el README.

Controles que evitaron conclusiones falsas (no son errores de la IA, pero conviene registrarlos):

- El test de fuga (decisión 14) se verificó metiendo una fuga a propósito para comprobar que el test falla. Un test que nunca falla no prueba nada.
- La comparación del modelo contra `corr_90d` a igual θ **no es justa**: el modelo predice correlaciones comprimidas hacia la media, genera grafos más ralos y usa menos canastas (3,3 contra 6,9 con θ = 0,5), lo que empeora su métrica (decisión 26). Y en una fecha de crisis el modelo casi no reacciona, mientras que la histórica sí. Esto salió al preguntarle a Claude qué debería pasar y contrastarlo con los datos.

10.7 Reflexión

Claude nos permitió llegar a un trabajo más completo del que habríamos hecho solos en el tiempo disponible: además de lo obligatorio, quedaron DSATUR, el polinomio cromático, Prim, la API y una interfaz interactiva. Nosotros nos ocupamos de planear, testear y revisar; la IA implementaba. Para no perder el control, pedimos explicaciones, hicimos que los algoritmos de grafos se implementaran a mano (los pide la materia) y los contrastamos con `networkx` en los tests.

Los errores que aparecieron fueron de criterio más que de sintaxis: un caso borde mal calculado, una interfaz confusa, un pedido incompleto que se ejecutó igual. Los atajaron los tests y la lectura crítica. Lo más difícil al trabajar con una IA no es la implementación, sino explicar bien el problema y la solución como los necesita el usuario. La IA no reemplaza saber qué se está midiendo ni poder defenderlo oralmente.

---

11. Limitaciones y extensiones

Limitaciones

- **El modelo mejora poco al baseline** (3 % de MAE con la mejor regresión, y el kNN elegido por validación empata). La mejora viene de los períodos de calma; en crisis, que es cuando más importa diversificar, ningún modelo le gana a `corr_90d`.
- **Con la fuente `modelo`, el grafo no reacciona a las crisis:** el número de canastas casi no cambia entre calma y crisis (3 a 4), mientras que con `corr_90d` (θ = 0,5) pasa de 7 en

enero de 2020 a 22 el 30 de marzo y 23 a fines de abril. La hipótesis de la propuesta ("en crisis aparecen más aristas y hacen falta más canastas") solo se cumple con la correlación histórica (sección 6).

- **Las canastas del modelo no superan a las de `corr_90d`.** Parte de la diferencia es un efecto de la comparación a igual θ (grafos más ralos, menos canastas), pero no se resolvió comparando a igual cantidad de canastas.
- **Universo chico y con sesgos:** 30 acciones que hoy están en el S&P 500 (sesgo de supervivencia); los sectores son los actuales (por ejemplo, GOOGL pasó a Comunicaciones en 2018); solo 28 de los 435 pares son del mismo sector, así que `mismo_sector` está desbalanceada.
- **Un solo período de test** (2021-2023). Las 33 fechas no son independientes, así que no hay significancia estadística.
- **Datos de `yfinance`:** no es una API oficial y su uso es personal o educativo.
- **El greedy es una heurística:** puede usar más canastas que χ(G). El polinomio cromático solo se calcula en grafos chicos (costo exponencial).
- **Una canasta no es una cartera optimizada:** no se asignan pesos de inversión ni se considera el retorno esperado. Diversificar por correlación reduce el riesgo, no garantiza ganancias.
- **Es un trabajo académico, no asesoramiento financiero.**

Posibles extensiones

- Comparar las fuentes a **igual cantidad de canastas**, o elegir θ por fuente.
- Probar el **clustering** como alternativa y medir la diferencia.
- Más acciones, y features adicionales (retornos sectoriales, variables macro, volumen).
- Modelos más ricos que los pedidos (por ejemplo, boosting) y probar valores de `k` mayores en kNN.
- **DSATUR con pesos**, y un criterio para elegir entre colores válidos que use $\hat{\rho}$ de forma más fina.
- Asignar pesos de inversión dentro de cada canasta.
- Predecir con modelos que reconozcan el régimen de mercado, para mejorar en crisis.

Anexo: reproducibilidad

Instrucciones completas de instalación y ejecución en el `README.md`. Para regenerar los resultados desde `backend/` con el entorno virtual activado:

```
python scripts/descargar_precios.py     # data/precios.csv
python -m scripts.construir_dataset     # data/dataset_pares.csv
python -m scripts.entrenar_modelo       # data/modelo.joblib,
data/metricas.json, figuras
python -m scripts.evaluar_canastas      # data/evaluacion_canastas.csv,
figura
python -m scripts.comparar_coloreos     #
documentacion/comparacion_coloreo.md
pytest                                   # tests, incluido el de leakage
```

Los notebooks `notebooks/01-dataset`, `02-split-y-baseline`, `03-modelos` y `04-analisis-de-errores` reproducen paso a paso el dataset, la partición, los modelos y el análisis de errores.

Documentos de apoyo: `PROPUESTA.md` (diseño del problema), `documentacion/decisiones.md` (28 decisiones con su justificación), `documentacion/preguntas-informe.md` (explicaciones en lenguaje simple) y `documentacion/evaluacion_canastas.md`.